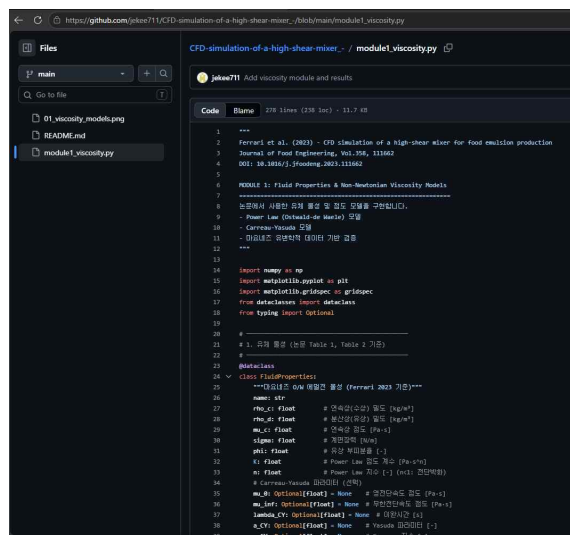


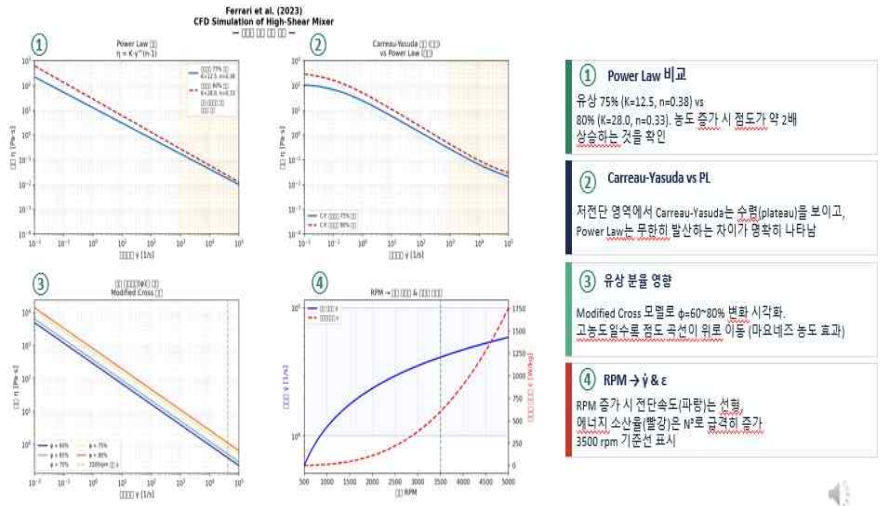
논문명 : CFD simulation of a high-shear mixer for food emulsion production

유튜브 링크 : <https://www.youtube.com/watch?v=pe9Sk86SSF8>

구현 결과 :



python module1_viscosity.py 실행 결과 → 01_viscosity_models.png 자동 생성



질문자	질문	답변
	마요네즈의 유상 함량이 5%p만 증가해도 점도가 약 2배 상승한다고 분석하셨습니다. 이를 캡스톤 프로젝트인 화장품 제조 공정에 적용할 때, 원료 배합 오차를 어느 정도 범위 내로 관리해야 공정 안정성을 확보할 수 있다고 예상하시나요?	화장품은 식품 대비 원료 종류가 많고 제형 스펙트럼이 넓다는 점에서 단순히 마요네즈 데이터를 그대로 적용하기 어렵습니다. 실무에서는 표준 점도 $\pm 10\%$ 를 스펙으로 설정합니다. 마요네즈 데이터(5%p → 점도 2배, %p당 약 14~15%)를 역산하면, $\pm 10\%$ 점도 스펙을 만족하기 위한 유상 함량 오차는 약 $\pm 0.7\%$ 이 내가 됩니다. 다만 화장품은 제형별 민감도가 크게 다릅니다. 저점도 제형(에센스·로션)은 점도 자체가 낮아 절대값 변동에 민감하여 더 엄격한 관리 필요합니다. 고점도 제형(영양크림·밤)은 폴리머·왁스 의존도가 커서 유상보다 증점제 농도·온도 이력이 점도에 더 큰 영향을 줍니다. 따라서 캡스톤 시뮬레이터에서는 유상 함량뿐 아니라 증점제 농도·교반 온도까지 변수로 포함해 $\pm 10\%$ 스펙 내 진입 가능한 운전 윈도우를 도출하는 방향이 적절해 보입니다.
강민규	RPM 증가에 따른 에너지 소산율이 N^3에 비례하여 비선형적으로 급증하는 것을 시각화하셨습니다. 과도한 발열을 방지하면서도 최적의 유화 품질을 얻을 수 있는 '공정 최적점'을 찾는 구체적인 기준은 무엇인가요?	현재 화장품 제조는 실험실 5,000 rpm × 5분 → 양산 가마 3,500 rpm × 5분으로 고정되어 있지만, 이는 경험적으로 정한 값일 뿐 검증된 최적점은 아닙니다. 마요네즈 논문 기준을 적용하면 최적점은 아래한 줄로 정리됩니다. $Ca/Cacr \geq 1.2$ 를 만족하면서, 온도 상승 15 °C 이내, d_{32}가 더 작아지지 않는 가장 낮은 RPM 즉 "깨지긴 하되, 발열 없이, 낭비 없는" 지점입니다. 현재 3,500 rpm은 "되긴 되는" 조건일 뿐, 같은 품질을 더 낮은 RPM에서 낼 수 있는 여지가 큼니다. 시뮬레이터로 후보 RPM을 좁힌 뒤 실험으로 검증하면, 발열·에너지 손실을 줄이면서 진짜 최적점을 찾을 수 있을 것이라고 생각합니다.
	현재 구현하신 Python 모듈을 향후 '제조 가마 시각화 시뮬레이터'의 코어 엔진으로 확장할 계획이라고 하셨습니다. 실제 공장 운영자가 이 시뮬레이터를 통해 사전에 예방할 수 있는 가장 대표적인 품질 불량 사례는 무엇이 될까요?	가장 대표적인 사례는 분리/크리밍(상분리)입니다. 보관 중 오일층이 떠오르거나 수상이 가라앉는 불량으로, 원인은 액적 크기(d_{32})가 충분히 작아지지 않아 시간이 지나며 Stokes 침강이 일어나는 것입니다. 화장품 클레임 중 가장 빈번하고, 출하 직후엔 정상으로 보이다 유통 단계에서 발견되어 회수 비용까지 발생하는 치명적 불량입니다. 시뮬레이터는 이를 출하 전에 예측할 수 있습니다. 가마 내부의 $Ca/Cacr$ 분포를 시각화하면 파쇄가 일어나지 않는 dead zone($Ca/Cacr < 1$ 영역)이 어디에 형성되는지 한눈에 보입니다. 운전자가 실제 배치를 돌리기 전에 RPM·임펠러 위치·교반 시간을 조정해 dead zone을 제거할 수 있어, "돌러보고 실패하는" 시행착오 없이 사전 예방이 가능합니다.

이서우	발표에서 세 가지 점도 모델(Power Law, Carreau-Yasuda, Modified Cross)을 모두 구현하셨는데, 실제 캡스톤 시뮬레이터에 이식할 때는 어떤 모델을 "기본 모델"로 채택할 계획이신가요? 또한 화장품 제형별로 최적 모델이 달라질 경우, 사용자가 모델을 선택할 수 있는 인터페이스를 설계할 의향이 있으신가요?	3. 기하학 파라미터 일반화 전략 논문의 <code>shear_rate_rotor_stator(R, gap)</code> 함수는 $R=45\text{mm}$, $gap=0.2\text{mm}$로 고정된 콘 밀 조건이므로, 캡스톤 시뮬레이터에서는 장비별 별도 함수가 아닌 프리셋 기반 일반화 구조로 확장할 예정입니다. 구체적으로는 VESSELS 객체에 <code>V-100/V-300/V-500</code> 등 사내 가마 프리셋을 등록하고, 각 프리셋에 <code>volume_L</code>, <code>D_v</code>, <code>H</code>, <code>D_imp</code> 외에 호모 기하학 파라미터(<code>R_rotor</code>, <code>gap</code>)를 추가합니다. 사용자가 가마 버튼을 선택하면 해당 프리셋 값이 자동 입력되도록 설계합니다.
	현재 구현에서 K, n, λ 등의 파라미터 값은 마요네즈 논문 기반으로 설정되어 있는 것으로 보입니다. 실제 화장품 제형에 적용하려면 제형별 레오미터 측정 데이터가 필요할 텐데, 해당 데이터를 어떻게 확보하실 계획인지, 그리고 파라미터 피팅은 어떤 방법(예: <code>scipy curve_fit</code> 등)으로 자동화할 예정인지 궁금합니다.	3. 기하학 파라미터 일반화 전략 논문의 <code>shear_rate_rotor_stator(R, gap)</code> 함수는 $R=45\text{mm}$, $gap=0.2\text{mm}$로 고정된 콘 밀 조건이므로, 캡스톤 시뮬레이터에서는 장비별 별도 함수가 아닌 프리셋 기반 일반화 구조로 확장할 예정입니다. 구체적으로는 VESSELS 객체에 <code>V-100/V-300/V-500</code> 등 사내 가마 프리셋을 등록하고, 각 프리셋에 <code>volume_L</code>, <code>D_v</code>, <code>H</code>, <code>D_imp</code> 외에 호모 기하학 파라미터(<code>R_rotor</code>, <code>gap</code>)를 추가합니다. 사용자가 가마 버튼을 선택하면 해당 프리셋 값이 자동 입력되도록 설계합니다.
	현재 <code>shear_rate_rotor_stator</code> 함수는 로터 반경 45mm, 간극 0.2mm로 고정된 논문의 믹서 조건을 기준으로 합니다. 캡스톤에서 다루는 제조 가마는 호모믹서와 패들믹서 등 장비 구조가 다를 수 있는데, 이 기하학 파라미터를 가변 입력으로 일반화할 계획이 있으신지, 아니면 장비별로 별도 함수를 작성할 예정이신지요?	3. 기하학 파라미터 일반화 전략 논문의 <code>shear_rate_rotor_stator(R, gap)</code> 함수는 $R=45\text{mm}$, $gap=0.2\text{mm}$로 고정된 콘 밀 조건이므로, 캡스톤 시뮬레이터에서는 장비별 별도 함수가 아닌 프리셋 기반 일반화 구조로 확장할 예정입니다. 구체적으로는 VESSELS 객체에 <code>V-100/V-300/V-500</code> 등 사내 가마 프리셋을 등록하고, 각 프리셋에 <code>volume_L</code>, <code>D_v</code>, <code>H</code>, <code>D_imp</code> 외에 호모 기하학 파라미터(<code>R_rotor</code>, <code>gap</code>)를 추가합니다. 사용자가 가마 버튼을 선택하면 해당 프리셋 값이 자동 입력되도록 설계합니다.
이학승	마요네즈랑 화장품 크림이 둘 다 비뉴턴 유체라고 하셨는데, 실제 화장품에도 지금 사용하는 점도 모델을 그대로 적용할 수 있는 건가요? 아니면 추가로 수정할 부분이 있나요?	비슷하지만 다른 부분이 있습니다. Power Law-Carreau-Yasuda 모델 자체는 비뉴턴 유체라면 공통으로 적용 가능합니다. 다만 마요네즈보다 화장품에 사용하는 원료의 양과 종류가 훨씬 다양하고, 그에 따라 유체 거동이 달라지기 때문에 이에 맞게 조절이 필요합니다. 1. 파라미터 재측정 : 마요네즈 기준값(K, n)을 화장품 제형의 레오미터 측정값으로 다시 피팅 2. 텍소트로피 : 화장품은 시간이 지나며 점도가 회복되는 거동이 강함 3. 온도 의존성 추가 : 가열(75°C)→냉각(30°C) 단계에서 점도가 크게 변하므로 $\eta(T)$ 항 필요
	RPM이 높아질수록 혼합은 잘 되지만 에너지 소산율도 급격히 증가한다고 하셨는데, 실제 공정에서는 적절한 RPM을 어떤 기준으로 정할 계획인가요?	$\varepsilon \propto N^3$ 관계상 표준 RPM값은 결국 "목표 품질을 달성하는 최저 RPM"으로 경험적으로 최적화된 값이며, 시뮬레이터는 이를 입력받아 가마 내부 거동을 재현합니다. 실제 제조 공정에서 단계별로 고정된 RPM 표준값이 이미 정해져 있어, 시뮬레이터에 이 값을 그대로 대입할 계획입니다. (투입 → 본 유화 → 냉각 단계별 프리셋)
	현재는 점도 모델 중심으로 구현하셨는데, 이후 실제 화장품 공정 데이터를 적용할 때 가장 중요하게 생각하는 변수는 무엇인지 궁금합니다. 예를 들어 RPM, 온도, 점도 중 어떤 변수가 가장 큰 영향을 줄 것이라고 예상하시나요?	품질을 직접 좌우하는 변수는 RPM입니다. 온도는 왁스 등의 고상 원료가 충분히 용해된 상태에서 유화가 진행되도록 하는 전제 조건 역할을 하며, 점도는 그 결과물이 제대로 나왔는지 확인하는 최종 품질 지표로 활용됩니다. 즉, RPM을 조절해 유화 상태를 만들고 → 최종 점도를 측정해 목표값에 도달했는지로 합격 여부를 판정할 예정입니다.

논문명 : BioSTEAM: A Fast and Flexible Platform for the Design, Simulation, and Techno-Economic Analysis of Biorefineries under Uncertainty

유튜브 링크 : <https://www.youtube.com/watch?v=UjvfuPCepLc>

```
print(f"총 설비투자비: ${tea.TCI:,.0f}")
```

```
... /usr/local/lib/python3.12/dist-packages/thermosteam/_stream.py:407: RuntimeWarning: <Stream: feed2> has been replaced in registry
  self._register(ID)
/usr/local/lib/python3.12/dist-packages/thermosteam/_stream.py:407: RuntimeWarning: <Stream: product2> has been replaced in registry
  self._register(ID)
/usr/local/lib/python3.12/dist-packages/IPython/core/interactiveshell.py:3553: RuntimeWarning: <MixTank: M2> has been replaced in registry
  exec(code_obj, self.user_global_ns, self.user_ns)
/usr/local/lib/python3.12/dist-packages/biosteam/_system.py:1185: RuntimeWarning: <System: cosmetic_process> has been replaced in registry
  self._register(ID)
```

```
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_2283/138666320.py in <cell line: 0>()
    11 sys.simulate()
    12
--> 13 tea = bst.TEA(sys, IRR=0.15, duration=(2026, 2046),
    14               depreciation='MACRS7', income_tax=0.21,
    15               operating_days=330)
```

```
TypeError: TEA.__init__() missing 10 required positional arguments: 'lang_factor', 'construction_schedule', 'startup_months', 'startup_FOCfrac', 'startup_VOCfrac', 'startup_salesfrac', 'WC_over_FCI', 'finance_interest', 'finance_years', and 'finance_fraction'
```

다음 단계: [오류 설명](#)

```
... === 공정 시뮬레이션 결과 ===
입력 유량: 115.0 kg/hr
출력 유량: 115.0 kg/hr
온도: 298.1 K
압력: 101325 Pa
MixTank: M1
ins...
[0] feed
  phase: 'l', T: 298.15 K, P: 101325 Pa
  flow (kmol/hr): Water    5.55
                   Ethanol  0.217
                   Glucose  0.0278
outs...
[0] s2
  phase: 'l', T: 298.15 K, P: 101325 Pa
  flow (kmol/hr): Water    5.55
                   Ethanol  0.217
                   Glucose  0.0278
/usr/local/lib/python3.12/dist-packages/thermosteam/_stream.py:407: RuntimeWarning: <Stream: feed> has been replaced in registry
  self._register(ID)
/usr/local/lib/python3.12/dist-packages/IPython/core/interactiveshell.py:3553: RuntimeWarning: <MixTank: M1> has been replaced in registry
  exec(code_obj, self.user_global_ns, self.user_ns)
```

구현 결과 :

질문자	질문	답변
강민규	BioSTEAM이 상용 소프트웨어인 Aspen Plus 대비 오차가 1% 이내라고 하셨습니다. 하지만 실행 결과 화면에서 TEA 모듈의 API 버전 호환 오류가 발생했는데, 이를 해결하기 위해 추가로 설정해야 하는 10개의 필수 인수 중 확장품 공정 경제성 평가에 가장 결정적인 변수는 무엇인가요?	가장 결정적인 변수는 lang_factor(랭 인수) 와 WC_over_FCI(운전자본 비율) 입니다. lang_factor는 장치 구매비용에서 총 설비투자비를 추산하는 배수로, 화장품 공정처럼 액체 처리 위주인 경우 보통 3.1~4.8 범위를 씁니다. WC_over_FCI는 운전자본 비율로 원료 재고와 제품 재고 비용을 반영합니다. 화장품은 원료 종류가 많고 재고 회전율이 중요하기 때문에 이 두 변수가 최종 MFSP, 즉 최소판매가격에 가장 큰 영향을 미친다고 판단합니다.
	Monte Carlo 시뮬레이션을 통한 불확실성 분석 기능을 강조하셨습니다. 화장품 제조 공정에서 원료 가격 변동이나 에너지 비용 상승 같은 리스크를 이 시뮬레이터로 분석했을 때, 실제 투자 의사결정에 어떤 정량적 도움을 줄 수 있나요?	Monte Carlo 시뮬레이션을 1,000회 이상 반복 실행하면 MFSP의 확률 분포를 얻을 수 있습니다. 예를 들어 원료 가격이 ±20% 변동할 때 MFSP의 90% 신뢰구간을 산출할 수 있어서, "최악의 경우에도 손익분기점 이상의 가격으로 판매 가능한가"를 수치로 판단할 수 있습니다. 이는 투자 리스크를 직관적으로 보여주기 때문에 실제 의사결정에서 "투자해도 되는가"를 근거 있게 판단할 수 있게 해줍니다.
	화장품 배합탱크 공정을 BioSTEAM의 MixTank 모듈로 모사할 수 있다고 제안하셨습니다. 유동의 물리적 거동뿐만 아니라 '소성로 열처리 공정'까지 확장이 가능하다면, 기초 화장품 외에 다른 확장품 제형(예: 색조, 파우더) 공정 시뮬레이션도 가능할까요?	기본적으로 가능하지만 추가 작업이 필요합니다. BioSTEAM의 MixTank는 액체 혼합 위주로 설계되어 있어서, 파우더처럼 고체-액체 혼합 공정은 커스텀 단위조작으로 직접 구현해야 합니다. 색조 제품의 경우 안료 분산 공정을 별도로 모델링해야 하고, 소성 공정은 HeatExchanger 모듈을 확장하면 모사 가능합니다. 완벽한 구현은 추가 개발이 필요하지만, 기본 틀은 BioSTEAM으로 충분히 구성할 수 있다고 생각합니다.
	발표 마지막에서 parameter sweep으로 최적화 가능하다고 하셨습니다.	화장품 공정에서 가장 효과적인 최적화 변수는 배합 온도와 교반 속도,

	데, 화장품 공정에서는 어떤 변수를 최적화 대상으로 두는 게 가장 효과적인 것 같다고 생각하셨는지 궁금합니다.	그리고 원료 투입 순서 라고 생각합니다. 배합 온도는 유화 안정성과 에너지 소비에 직결되고, 교반 속도는 혼합 균일도와 제품 품질에 가장 큰 영향을 미칩니다. BioSTEAM에서는 이 변수들을 루프로 돌리면서 출력 조성의 균일도와 에너지 소비량을 동시에 평가하는 다목적 최적화로 접근하면 효과적인 것 같습니다.
이학승	화장품은 점도가 계속 변하는 비뉴턴 유체가 많은데, BioSTEAM에서도 이런 점도 변화를 반영할 수 있는지 아니면 일정 점도로 가정하고 계산하는 방식인지 궁금합니다.	현재 BioSTEAM의 기본 점도 모델은 뉴턴 유체를 가정하고 있어서, 크림이나 로션처럼 전단 속도에 따라 점도가 변하는 비뉴턴 유체를 직접 반영하기는 어렵습니다. 다만 BioSTEAM은 커스텀 단위조작 클래스를 만들 수 있기 때문에, Power Law 모델이나 Bingham Plastic 모델을 직접 구현해서 적용하는 것은 가능합니다. 이 부분은 저희 캡스톤 프로젝트에서 추가로 개발해야 할 부분으로 인식하고 있습니다.
	현재 발표에서는 입력 조성 변화에 따른 결과 예측 중심으로 설명해주셨는데, 저희가 진행하는 프로젝트는 화장품 호모믹서 시뮬레이터를 기반으로 한 디지털 트윈 구현을 목표로 하고 있습니다. 혹시 BioSTEAM도 RPM, 온도, 점도 같은 실시간 공정 데이터를 연동해서 실시간 공정 시뮬레이션 형태로 확장 가능한지 궁금합니다.	현재 BioSTEAM은 정상 상태 즉 steady-state 시뮬레이션만 지원하기 때문에 실시간 연동은 기본 기능으로는 어렵습니다. 하지만 Python 기반이기 때문에 외부 데이터를 실시간으로 받아서 BioSTEAM 파라미터를 주기적으로 업데이트하는 방식으로 유사 실시간 시뮬레이션을 구현하는 것은 가능합니다. 예를 들어 센서 데이터를 받아서 매 1분마다 시뮬레이션을 재실행하는 방식으로 디지털 트윈과 연계하는 구조를 생각해볼 수 있습니다.
유재경	BioSTEAM이 상용 소프트웨어인 Aspen Plus 대비 $\pm 1\%$ 이내 정확도를 가진다고 논문에서 검증했다고 하셨는데, 그럼에도 불구하고 산업 현장에서 여전히 Aspen Plus 같은 유료 소프트웨어를 쓰는 이유가 무엇이라고 생각하시나요? 무료·오픈소스 도구의 한계는 어떤 부분일까요?	가장 큰 이유는 신뢰성 보증과 기술 지원입니다. Aspen Plus는 수십 년간 산업 현장에서 검증된 레퍼런스가 쌓여 있고, 문제 발생 시 공식 기술 지원을 받을 수 있습니다. 반면 BioSTEAM 같은 오픈소스는 버전 업데이트 시 API가 갑자기 바뀌는 것처럼 안정성 보장이 어렵고, 문제 발생 시 직접 해결해야 합니다. 또한 기업 내부 보안 정책상 외부 오픈소스 도구 사용이 제한되는 경우도 있습니다. 이런 한계에도 불구하고 초기 설계 단계나 학술 연구에서는 BioSTEAM이 충분히 경쟁력 있다고 생각합니다.
	화장품 배합탱크 공정을 BioSTEAM의 MixTank 모듈로 모사할 수 있다고 하셨는데, 화장품은 원료 종류가 수십 가지에 달하고 일부는 BioSTEAM의 기본 화학물질 데이터베이스에 없을 가능성이 높습니다. 이런 경우 어떻게 시뮬레이션을 구성해야 할까요?	BioSTEAM은 thermosteam이라는 열역학 라이브러리를 기반으로 하는데, 여기에 없는 물질은 두 가지 방법으로 처리할 수 있습니다. 첫째, bst.Chemical 클래스에서 분자량, 끓는점, 밀도, 점도 같은 물성값을 직접 입력해서 커스텀 화학물질을 정의할 수 있습니다. 실제로 화장품 원료의 물성 데이터는 원료 공급사의 데이터시트나 문헌에서 찾을 수 있습니다. 둘째, 구조가 비슷한 대리 물질을 대신 사용하는 방법도 있습니다. 예를 들어 특정 계면활성제가 없다면 비슷한 구조의 지방산 에스터로 근사할 수 있습니다.
	BioSTEAM은 원래 바이오리파이너리(바이오 연료, 바이오 화학물질 생산) 공정을 위해 개발된 도구인데, 화장품 제조처럼 점성이 매우 높고 비뉴턴 유체 특성을 가지는 공정에도 적합할까요? 어떤 부분이 추가로 필요할 것 같습니까?	솔직히 말씀드리면 BioSTEAM은 화장품 공정에 바로 적용하기에는 한계가 있습니다. 바이오리파이너리는 주로 저점도 액체 공정이 중심인 반면, 화장품은 고점도 비뉴턴 유체, 유화 공정, 안료 분산 등 특수한 공정이 많습니다. 추가로 필요한 부분은 크게 세 가지입니다. 첫째 비뉴턴 점도 모델 구현, 둘째 유화 공정 전용 단위조작 모듈 개발, 셋째 고체 원료 처리 모듈 강화입니다. BioSTEAM의 틀을 활용하되 화장품 특화 모듈을 추가 개발하는 방향이 현실적이라고 생각합니다.

논문명 : kh2d-solver: A Python Library for Idealized Two-Dimensional Incompressible Kelvin-Helmholtz Instability

유튜브 링크 : <https://www.youtube.com/watch?v=F4vQkZJ3zog>

구현 결과 :

```
%cd /content
!rm -rf suppl_kh2d-solver_paper
!rm -rf kh2d_generated_outputs

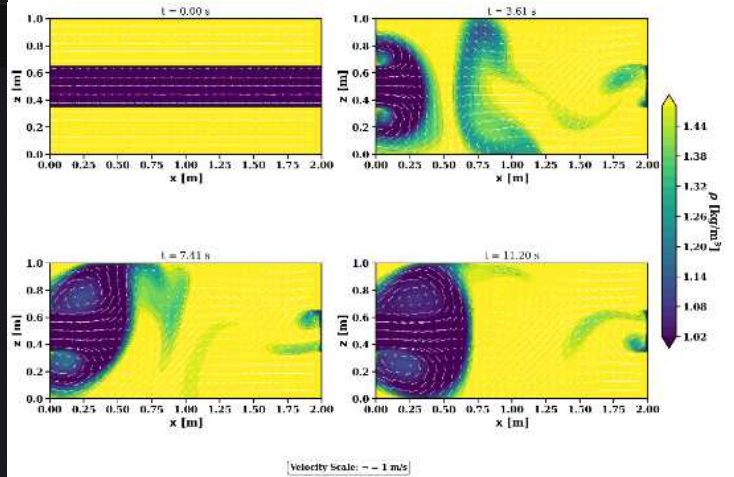
!pip install -q kh2d-solver numpy scipy xarray matplotlib netCDF4 tqdm pandas

/content

# =====
# 2. kh2d-solver의 사용 case
# =====
import numpy as np
from kh2d_solver import (
    KH2DSolver,
    ShearLayer,
    DoubleShear,
    RotatingShear,
    ForcedTurbulence,
    DataHandler
)

np.random.seed(0)

cases = [
    {
        "name": "basic_shear",
        "scenario name": "Basic Shear Layer",
        "solver": dict(nx=156, nz=128, lx=2.0, lz=1.0),
        "ic_class": ShearLayer,
        "ic_params": dict(
            shear_thickness=0.05,
```



질문자	질문	답변
	발표에서 KH 불안정성이 대기 구름이나 해양 해류 혼합의 핵심 메커니즘이라고 하셨는데, 제안하신 2D 모델이 3차원적인 실제 자연 현상의 혼합 효율을 어느 정도 수준까지 정밀하게 모사할 수 있다고 보시나요?	실제 대기나 해양의 KH 불안정성은 3차원 난류 구조를 가지기 때문에, 이 논문에서 사용한 2D 모델만으로 실제 자연 현상을 완전히 똑같이 재현하기는 어렵습니다. 하지만 KH instability의 초기 단계에서는 소용돌이가 말려 올라가는 기본 구조가 주로 2차원 형태로 나타나기 때문에, 혼합이 시작되는 과정이나 와류가 형성되는 메커니즘 자체는 2D 모델로도 충분히 관찰할 수 있습니다. 따라서 이 논문은 실제 자연계의 절대적인 혼합량을 정확히 예측하기보다는, 조건에 따라 mixing pattern이나 instability 구조가 어떻게 달라지는지를 비교하고 분석하는 데 의미가 있다고 볼 수 있습니다. 다만 실제 환경에서는 시간이 지나면서 더 복잡한 3차원 난류가 발생하기 때문에, 이를 정밀하게 모사하려면 향후에는 3차원 모델로 확장할 필요가 있다고 생각합니다.
강민규	Numba JIT 기법을 적용해 C언어 수준의 성능을 확보하셨다고 설명하셨습니다. 만약 향후 연구 과제인 3차원 모델로 확장할 경우, 일반 데스크톱 환경에서도 여전히 실시간에 가까운 시뮬레이션 속도를 유지할 수 있을까요?	Numba JIT를 사용하면 Python 코드의 반복 계산을 빠르게 만들 수 있기 때문에 2D 모델에서는 일반 데스크톱에서도 비교적 효율적으로 계산할 수 있습니다. 하지만 3D로 확장하면 격자 수와 계산량이 급격히 증가하기 때문에, 같은 방식으로 실시간에 가까운 속도를 유지하기는 어렵다고 볼 수 있습니다. 특히 3D에서는 속도장, 압력장, 밀도장 등을 더 많은 격자에서 계산해야 하고, 압력 Poisson equation 계산도 훨씬 복잡해집니다. 따라서 Numba JIT는 여전히 성능 향상에는 도움이 되지만, 3D 모델을 실시간 수준으로 구현하기 위해서는 추가적인 계산 최적화 방법이 더 필요하다고 생각합니다.
	Shannon Entropy를 통해 유동의 무질서도를 정량화하셨는데, 이 지표를 실제 산업 현장의 유체 혼합 공정(예: 화장품 교반)에 적용했을 때 기존의 평균 데이터 분석 방식보다 어떤 구체적인 이점을 얻을 수 있을까요?	기존의 평균 속도나 평균 농도 같은 방식은 전체적인 경향은 볼 수 있지만, 실제로 유체 내부에서 얼마나 균일하게 섞이고 있는지를 국소적으로 복잡한 혼합 구조가 어떻게 형성되는지는 잘 보여주지 못하는 경우가 많습니다. 반면 Shannon Entropy는 유동장의 무질서도나 복잡성을 정량적으로 표현할 수 있기 때문에, 혼합이 진행되면서 유체가 얼마나 균일해지고 있는지를 더 민감하게 확인할 수 있다는 장점이 있습니다. 예를 들어 화장품 교반 공정에서는 단순 평균값만으로는 dead zone이나 국소적인 미혼합 영역을 파악하기 어려울 수 있는데, entropy 기반 분석을 사용하면 이런 공간적인 mixing quality 변화를 더 잘 추적할 수 있을 것으로 생각합니다.
	Forced turbulence case가 가장 높은 Reynolds 수 조건임에도 mixing efficiency가 가장 낮게 나타났다고 하셨는데, 이것이 단순히 외부 forcing이 유동 구조를 교란하기 때문인지, 아니면	논문 결과를 보면 forced turbulence case에서는 Reynolds 수는 가장 높았지만, 외부 forcing이 계속 들어오면서 유동이 매우 불규칙한 상태를 유지하게 됩니다. 그래서 에너지가 큰 소용돌이를 만들어 밀도

이서우	quasi-equilibrium 상태에서 에너지 입력과 소산이 균형을 이루어 실질적인 밀도 혼합이 억제되기 때문인지, 물리적 메커니즘을 좀 더 구체적으로 설명해 주실 수 있나요?	층을 크게 뒤집는 데 사용되기보다는, 작은 난류 구조를 계속 만드는 데 소비되는 경향이 나타난 것으로 볼 수 있습니다. 즉 난류 세기가 크다고 해서 반드시 mixing efficiency가 높아지는 것은 아니고, 오히려 너무 불규칙한 상태가 유지되면서 실제 밀도 혼합은 비효율적으로 진행된 것으로 해석할 수 있습니다. 반면 double shear layer case에서는 비교적 큰 와류 구조가 안정적으로 형성되면서 밀도층을 더 효과적으로 섞을 수 있었던 것으로 보입니다.
	후처리 단계에서 대규모 데이터를 처리할 때 최대 10,000개로 랜덤 샘플링을 제한하셨는데, 전단층 경계면처럼 공간적으로 불균일하게 분포하는 데이터에서 랜덤 샘플링이 entropy나 complexity index 계산 결과에 편향을 유발할 가능성은 없는지, 그리고 이를 검증하셨다면 어떤 방식으로 확인하셨나요?	논문에서처럼 전체 데이터가 매우 클 때 10,000개만 랜덤으로 뽑으면 계산 속도를 줄일 수 있다는 장점은 있습니다. 하지만 전단층 경계면이나 강한 와류처럼 특정 위치에만 중요한 구조가 몰려 있는 경우에는, 랜덤 샘플링 과정에서 그 부분이 충분히 선택되지 않을 가능성이 있습니다. 그러면 entropy나 complexity index가 실제보다 낮게 계산될 수도 있습니다. 그래서 샘플 개수를 바꿔도 결과가 비슷하게 나오는지 확인하거나, 영역을 나누어서 고르게 샘플링하는 방식으로 추가 검증이 필요하다고 생각합니다.
	발표에서 Boussinesq 근사를 적용하여 밀도 차이가 작은 경우로 제한하셨는데, 화장품 유화 공정처럼 오일상과 수상의 밀도 차이가 상대적으로 크고 계면활성제까지 포함된 실제 계에서는 이 근사가 더 이상 유효하지 않을 수 있습니다. 이런 경우 non-Boussinesq 조건으로 모델을 확장하려면 현재 solver 구조에서 어떤 부분을 수정해야 한다고 생각하시나요?	Boussinesq 근사는 기본적으로 밀도 차이가 크지 않다고 가정하는 모델인데, 화장품 유화 공정처럼 오일과 물이 함께 섞이는 경우에는 밀도 차이도 더 크고 두 유체 사이의 경계면 움직임도 중요하기 때문에 그대로 적용하기는 어려울 수 있습니다. 그래서 이런 경우에는 현재처럼 밀도를 거의 일정하다고 두는 방식이 아니라, 위치에 따라 밀도가 계속 달라지도록 계산해야 합니다. 또 단순히 유체가 섞이는 것뿐 아니라 오일방울이 깨지거나 합쳐지는 현상, 계면 움직임 같은 것도 함께 고려해야 하기 때문에 현재 solver보다 더 복잡한 two-phase flow 형태로 확장해야 할 것 같습니다.
유재경	화장품 제조 공정에서도 서로 다른 점도와 밀도를 가진 액체(예: 오일상과 수상)를 섞어 에멀전을 만드는 과정이 있는데, 이 kh2d-solver처럼 KH 불안정성을 분석하는 접근이 화장품 혼합 공정 시뮬레이터에 어떻게 도움이 될 수 있을까요?	화장품 호모믹서에서도 오일상과 수상처럼 서로 다른 밀도와 점도를 가진 유체가 고속 회전에 의해 만나면서 계면에서 강한 shear가 발생합니다. 이 과정에서 KH instability처럼 계면이 흔들리며 vortex와 mixing layer가 형성될 수 있습니다. 타겟 논문은 이러한 shear 기반 mixing 현상을 수치적으로 해석하고, vorticity와 density field를 통해 mixing 구조를 시각화했다는 점에서 의미가 있습니다. 따라서 이 접근을 화장품 공정에 적용하면 RPM 변화에 따른 mixing 패턴, dead zone 발생 위치, 에멀전 형성 과정, mixing uniformity 등을 디지털 트윈 형태로 분석하고 시각화하는 데 도움이 될 수 있다고 생각합니다.
	발표에서 kh2d-solver가 Python 기반인데도 Numba JIT 컴파일을 통해 C 수준의 속도를 낼 수 있다고 하셨는데, 직접 실행해보셨을 때 체감상 속도가 어땠나요? 그리고 왜 굳이 C/C++가 아닌 Python으로 solver를 만드는 게 연구에 유리한가요?	직접 실행했을 때 Python 코드임에도 불구하고 생각보다 무겁지 않게 돌아갔습니다. 논문에서도 Numba JIT를 사용하면 반복 계산 부분이 기계어 수준으로 최적화되어 순수 Python보다 약 10-50배 정도 빨라질 수 있다고 설명합니다. Python으로 solver를 만드는 장점은 코드 수정과 실험 조건 변경이 쉽다는 점입니다. 예를 들어 격자 크기, 초기 조건, Reynolds number, 시각화 방식을 빠르게 바꿔가며 테스트할 수 있습니다. C/C++보다 실행 속도는 불리할 수 있지만, 연구 초기 단계나 디지털 트윈 프로토타입 구축에서는 빠른 구현, 가독성, 데이터 분석·시각화 연동 측면에서 Python이 더 유리하다고 생각합니다.
	본 연구에서 3차원이 아닌 2차원 모델로 단순화한 이유를 Spanwise 방향 변화가 작기 때문이라고 설명하셨는데, 만약 화장품 제조처럼 교반기로 강하게 휘저어 3차원 난류가 지배적인 공정에서는 이 2D 모델을 그대로 적용하기 어려울까요? 그렇다면 어떤 부분이 바뀌어야 할까요?	네, 그대로 적용하기는 어렵다고 보는 게 맞습니다. 논문에서는 KH 불안정성의 가장 빠르게 성장하는 모드가 2차원적이라는 점을 근거로 2D 단순화를 사용했습니다. 하지만 화장품 호모믹서는 임펠러 회전, 축방향 순환, 벽면 효과, rotor-stator 주변 난류가 모두 3차원적으로 나타납니다. 그래서 2D 모델은 초기 프로토타입이나 계면 mixing 경향을 보는 데는 쓸 수 있지만, 실제 공정 예측에는 한계가 있습니다. 실제 적용하려면 3D 격자, 회전 교반기 형상, 비뉴턴 점도 모델, 자유계면 또는 다상유동 모델, 그리고 RPM에 따른 시간 의존 경계조건이 추가되어야 합니다. 즉, 이 논문은 완성형 호모믹서 모델이라기보다 3D 디지털 트윈으로 확장하기 위한 기초 solver 구조로 보는 게 적절합니다.

논문명 : PySPH: a Python-based framework for smoothed particle hydrodynamics

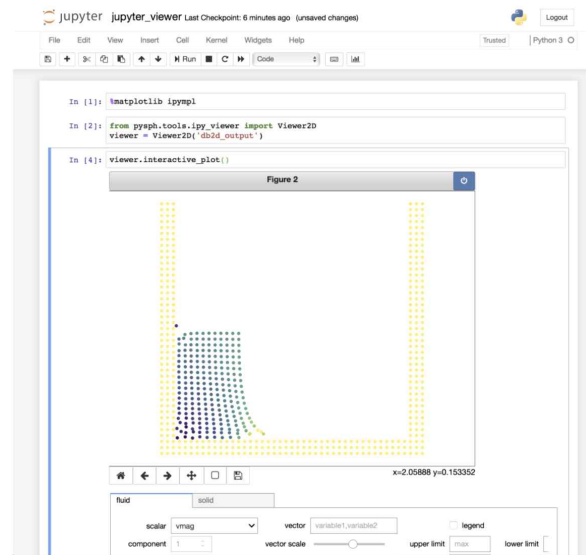
유튜브 링크 : <https://www.youtube.com/watch?v=Y45w4RF0n-w>

구현 결과 :

```

1 """ An implementation of a general solver base class """
2 from future import print_function
3 # System library imports.
4 import os
5 import numpy
6
7 from compyle.profile import profile, profile_ctx
8 # PySPH imports
9 from pysph.base.kernels import CubicSpline
10 from pysph.sph.acceleration_eval import make_acceleration_evals
11 from pysph.sph.sph_compiler import SPHCompiler
12
13 from pysph.solver.utils import ProgressBar, load, dump
14
15 import logging
16 logger = logging.getLogger(__name__)
17
18 EPSILON = numpy.finfo(float).eps*2
19
20
21 class Solver(object):
22     """Base class for all PySPH Solvers
23     """
24     def __init__(self, dim=2, integrator=None, kernel=None,
25                  n_damp=0, t=1.0, dt=1e-3,
26                  adaptive_timestep=False, cfl=0.3,
27                  output_at_times=(),
28                  fixed_h=False, **kwargs):
29         """**Constructor**
30
31         Any additional keyword args are used to set the values of any
32         of the attributes.
33
34         Parameters
35     """

```



질문자	질문	답변
이서우	발표에서 Neighbor Search가 입자 수가 많아질수록 전체 성능을 거의 결정한다고 하셨는데, Grid 기반과 Octree 기반 두 방법을 직접 실험하셨을 때 어느 시점부터 성능 차이가 유의미하게 나타났나요? 그리고 확장품 제조 가마처럼 대용량·점성 유체를 시뮬레이션하려면 현실적으로 몇 개 수준의 입자 수가 필요하고, 일반 데스크탑 환경에서 감당 가능한 수준인지 궁금합니다	Neighbor Search 알고리즘은 입자 수가 적을 때는 단순한 Linked List 방식이 캐시 효율 면에서 유리하지만, 입자 분포가 불균일해지거나 지지 반경이 변하는 시점부터는 Octree 기반 방식이 유의미한 성능 우위를 점하게 됩니다. 일반적인 데스크탑 환경에서는 PySPH의 내장 뷰어를 통해 약 500만 개 수준의 입자까지 인터랙티브하게 시각화하고 다룰 수 있도록 최적화되어 있으며, 이는 대용량 유체 시뮬레이션을 감당하기에 충분한 수준입니다. 특히 PySPH는 입자 수가 많아질수록 성능을 높이기 위해 OpenMP(멀티코어)와 GPU(Z-Order SFC, Octree) 가속 알고리즘을 지원하여 일반 데스크탑에서도 고성능 계산이 가능하도록 설계되었습니다.
	PEC (Predict-Evaluate-Correct) Integrator 구조를 설명하셨는데, 각 time step의 크기(dt)는 어떤 기준으로 결정되나요? 점성이 매우 높은 유체를 다룰 때 dt가 너무 작아져 계산 시간이 폭발적으로 증가하는 문제가 생길 수 있는데, PySPH에서 이 문제를 어떻게 제어하는지 코드에서 확인하셨나요?	PySPH에서 각 시간 단계(dt)의 크기는 물리적 안정성을 보장하기 위해 수치 해석적 안정성 기준인 CFL(Courant-Friedrichs-Lowy) 조건을 바탕으로 결정됩니다. 점성이 매우 높은 유체를 다룰 때 dt가 작아져 계산 시간이 급증하는 문제를 제어하기 위해, PySPH는 Matrix-free formulation을 이용한 반복적인 행렬 풀이 방식을 지원합니다. 또한, Iterated Groups 기능을 통해 한 타임 스텝 내에서 특정 방정식이 수렴할 때까지 반복 계산을 수행함으로써 시뮬레이션의 안정성을 유지하면서도 전체적인 계산 효율을 높일 수 있는 구조를 코드상에서 제공합니다.
	Python 코드가 내부적으로 Cython으로 자동 변환되어 실행되는 구조라고 하셨는데, 실제로 코드를 실행하다가 오류가 발생했을 때 어느 레이어에서 문제가 생긴 건지 추적하기 어렵지 않았나요? 사용자가 물리 모델을 커스터마이징할 때 이 자동 변환 과정이 오히려 디버깅의 장벽이 될 수 있다고 생각하시는지 궁금합니다.	사용자가 작성한 순수 Python 코드가 내부적으로 Cython이나 GPU용 C 코드로 자동 변환되는 과정은 성능을 극대화하지만, 내부 레이어의 복잡성으로 인해 디버깅 시 초기 장벽이 될 수 있습니다. 특히 변환되는 함수 내부에서는 NumPy 객체나 Python 객체를 생성할 수 없는 등의 엄격한 제약이 따르는데, 이를 위반할 경우 변환 과정에서 오류가 발생할 수 있습니다. 하지만 PySPH는 Mako 템플릿을 활용해 코드 생성 로직을 단순화하고 정형화된 구조를 제공함으로써, 사용자가 물리 법칙 구현에 집중할 수 있도록 돕고 자동 변환에 따른 디버깅의 어려움을 최소화하려 노력했습니다.
이학승	현재 저희 프로젝트처럼 확장품 유화 크림과 같은 비뉴턴 유체를 모델링하려면 PySPH를 바로 적용할 수 있는지, 아니면 추가로 수정해야 하는 부분이 있는지 궁금합니다.	화장품 유화 크림과 같은 비뉴턴 유체를 모델링하기 위해 PySPH를 활용하는 것은 충분히 가능하지만, 기본 모델 외에 사용자의 추가적인 정의가 필요합니다. PySPH는 기본적으로 압축성/비압축성 유체뿐만 아니라 탄성 역학(Elastic dynamics)과 강체 역학(Rigid body dynamics)을 모두 지원하는 범용 프레임워크입니다. 따라서 비뉴턴 유체의 특성을 반영하기 위해서는 전단 속도에 따라 점도가 변하는 Constitutive Equation(구성 방정식)을 사용자가 직접 Equation 클래스로 작성하여 프레임워크에 주입해야 하며, PySPH는 이러한 확장이 용이하도록 설계되어 있습니다.

이학승	발표에서 SPH는 자유표면이나 유체 계면 변화 표현에 장점이 있다고 설명해주셨는데, 실제 화장품 호모믹서처럼 강한 전단을 받는 비뉴턴 유체 혼합 공정에서도 Grid 기반 CFD보다 더 적합하다고 볼 수 있을까요? 그렇다면 그 이유는 무엇인가요?	SPH는 격자가 없는 Meshless(격자 없음) 및 Lagrangian 방식이므로, 화장품 호모믹서 공정처럼 유체 계면이 급격하게 변하고 대변형이 발생하는 상황에서 Grid 기반 CFD보다 훨씬 적합합니다. 격자 기반 방식은 복잡한 경계나 자유표면을 처리할 때 격자가 찢그러지는 문제가 발생할 수 있지만, SPH는 입자 자체가 물리량을 운반하며 자유롭게 움직이기 때문에 물리적 불연속면을 더 자연스럽게 묘사할 수 있습니다. 이러한 특성 덕분에 강한 전단을 받는 비뉴턴 유체의 복잡한 유동 패턴을 포착하는 데 더 유리한 구조를 가집니다.
	실제 화장품 호모믹서에서는 RPM이 올라갈수록 점도와 유동 패턴이 계속 변하는데, SPH 방식에서도 이런 공정 조건 변화를 실시간으로 반영할 수 있을까요?	실제 화장품 호모믹서 공정에서 RPM 변화에 따른 유동 패턴 변화는 SPH 방식 내에서 효과적으로 반영될 수 있습니다. PySPH는 Rigid body dynamics(강체 역학) 모듈을 지원하므로, 회전하는 임펠러를 강체 입자로 모델링하고 이에 가해지는 각속도나 물리적 조건을 시뮬레이션 도중에 업데이트할 수 있습니다. 또한, Iterated Groups나 사용자 정의 콜백 함수를 활용하면 시간에 따라 변화하는 점도나 경계 조건을 실시간으로 계산에 반영하여 공정의 역동적인 변화를 묘사할 수 있습니다.
유재경	SPH는 격자(mesh) 없이 입자만으로 유체를 표현한다고 하셨는데, 기존 CFD 방식과 비교했을 때 어떤 상황에서 SPH가 더 유리하고, 반대로 어떤 경우에는 불리할까요?	기존 격자 기반 CFD와 비교했을 때, SPH는 복잡한 기하학적 구조 내의 자유표면 유동이나 격자 생성이 어려운 대변형 시뮬레이션 상황에서 압도적으로 유리합니다. 반면, SPH는 인접한 입자들을 찾는 연산(Neighbor search)이 필수적이며 입자 수가 많아질수록 계산 비용이 격자 방식보다 높게 발생할 수 있다는 단점이 있습니다. 또한, PySPH의 경우 현재 멀티코어 CPU(OpenMP) 성능은 매우 성숙했으나 GPU 최적화는 아직 최적화 여지가 남아 있어, 극도로 정밀한 계산이 필요한 일부 상황에서는 계산 효율을 신중히 고려해야 합니다.
	화장품 제조 공정에서도 크림이나 로션 같은 유체를 다루는데, PySPH 같은 입자 기반 시뮬레이션을 화장품 제조 시뮬레이터에 적용한다면 어떤 부분에서 도움이 될 수 있을까요?	PySPH와 같은 입자 기반 시뮬레이션을 화장품 제조 공정에 도입하면 혼합 공정의 효율(Mixing efficiency)을 정량적으로 분석하고 최적화하는 데 큰 도움을 줄 수 있습니다. 특히 가마 내부에서 유체가 섞이지 않는 사각지대(Dead zone)를 시각화하여 설비를 개선하거나, 실험적으로 확인하기 어려운 미세한 유동 패턴을 예측함으로써 불량률을 줄일 수 있습니다. 무엇보다 PySPH는 재현 가능한 연구(Reproducible research)를 지원하므로, 제조 레시피별 시뮬레이션 데이터를 자산화하여 신제품 개발 기간을 단축하는 데 기여할 수 있습니다.
	화장품은 물보다 훨씬 점성이 높고 끈적한 유체인데, PySPH가 지원하는 SPH 모델(WCSPH, ISPH 등) 중 이런 점성 유체를 다루기에 어떤 게 더 적합할 것 같다고 생각하세요?	화장품과 같이 점성이 높고 비압축성 특성을 가진 유체에는 PySPH가 제공하는 모델 중 TVF(Transport Velocity Formulation)나 ISPH(Incompressible SPH) 계열이 가장 적합합니다. TVF는 입자의 뭉침 현상을 방지하고 입자 분포를 균일하게 유지하여 비압축성 유동을 정확하게 계산하는 데 강점이 있으며, ISPH 모델은 압력 포아송 방정식을 풀어 밀도 변동을 엄격하게 제어하므로 끈적한 유체의 물리적 거동을 묘사하기에 유리합니다. 특히 PySPH는 이러한 다양한 스킴을 코드 수정 없이 선택하여 실행할 수 있는 SchemeChooser 기능을 제공하여 최적의 모델을 찾는 과정을 돕습니다.